

IA Aplicada com LLMs

Aula 08 — Checkpoint não é memória

A fronteira

	Checkpoint (nota 01 §8)	Memória (daqui em diante)
Escopo	uma execução	todas as execuções
Propósito	retomar	lembrar
Forma	estado íntegro	fragmento selecionado
Leitura	uma vez, no início	por relevância, a cada volta
Crescimento	não cresce	cresce sem teto
Descarte	ao concluir a execução	decisão de projeto

A linha decisiva é a terceira.

A questão que abre a aula

Examina-se a proposição: **serializar o Estado** de todas as execuções produz memória.

10 execuções · 36 passos · 21.360 tokens
contexto necessário para incluir tudo: ~5.340 tokens

A proposição é falsa: o que se obtém é arquivo morto. Memória exige recuperar, e recuperar exige decidir o que é relevante agora.

É a Aula 07 apontada para dentro: em vez de indexar documentos, indexar **episódios** — o registro do que o agente fez numa execução anterior.

O arquivo morto e a memória, lado a lado

ARQUIVO MORTO	10 execuções serializadas contexto necessário: ~5.340 tokens
MEMÓRIA	3 execuções recuperadas por relevância contexto necessário: ~80 tokens

As três recuperadas são **as duas de Lisboa e uma de refeição** — exatamente as que a pergunta atual exige.

A diferença não é de compressão. É de **seleção**.

A mesma execução, nas duas formas

	<i>checkpoint</i>	memória
Tamanho	~250 tokens	~64 tokens
Campo <code>passos</code>	a lista , com argumentos e resultados	o número 3
Serve para	reconstruir onde a execução estava	dimensionar o que ela custou

A memória descartou ~75% do *checkpoint* — **de propósito. Um guarda o *que aconteceu*; o outro guarda *que aconteceu*.**

Os dois têm um campo chamado `passos`, e não é o mesmo campo. Comparar estruturas por nome de campo não distingue as duas; tentar retomar, sim.

A mesma pergunta às duas. Duas vezes.

	<i>"retome a <code>exec-0a1f</code> onde parou"</i>	<i>"o que ajuda na despesa de agora?"</i>
<i>checkpoint</i>	RESPONDE — passos, ferramentas, histórico, término	não: seria ler as dez inteiras, ~5.340 tokens
memória	não: guarda <i>que</i> aconteceu, não o <i>que</i>	RESPONDE — 3 fragmentos, ~80 tokens

Cada uma falha na pergunta da outra, e falha pela mesma razão: cada uma guardou o que a **sua** pergunta exige.

O checkpoint da nota 01 não é uma memória ruim. É outra coisa — e é daqui em diante que a memória é construída.

E a terceira estrutura: o RAG

Os três guardam texto, são lidos antes de decidir, podem usar o **mesmo índice vetorial**. Divergem na **origem**:

	<i>checkpoint</i>	memória	RAG
Pergunta	onde esta execução parou	o que o agente já viveu	o que está escrito
Quem escreveu	o agente, agora	o agente, antes	terceiros
Escopo	uma execução	todas as execuções	o corpus
Contradição é	impossível	normal	falha de curadoria
Onde	nota 01 §8	esta aula	Aula 07

No RAG existe **alguém responsável** pelo que está no corpus. Na memória, o autor

Parte 1

Curto e longo prazo

A fronteira tem nome na literatura

	<i>short-term</i> (memória de trabalho)	<i>long-term</i>
Alcance	a interação corrente	todas as sessões
Implementação	<i>rolling buffer</i> , a janela de contexto	banco, grafo, índice vetorial
Ao fim da sessão	não sobrevive	persiste

A divisão é **anterior** a qualquer taxonomia: vem antes de episódica, semântica e procedural.

IBM (2025): a de curto prazo "não retém informação além da sessão, o que a torna inadequada para personalização ou aprendizado de longo prazo".

(tradução livre)

O mapeamento — a aula já construiu as duas metades

Literatura	Nesta aula
<i>short-term</i>	a janela: <i>system prompt</i> , objetivo, trajetória
— sua persistência	o checkpoint (nota 01 §8)
— sua compressão	<i>tool clearing, compaction</i> (nota 03 §5)
<i>long-term</i>	as três memórias (a seguir)
— <i>extraction</i>	as três políticas de escrita (nota 03 §1)
— <i>consolidation</i>	contradição, decaimento (nota 04 §1–2)
— <i>retrieval</i>	o orçamento de contexto (nota 03 §4)

Episódica, semântica e procedural são **subtipos do longo prazo** — não uma lista plana de três.

E o checkpoint, renomeado

Redis (2026), sobre a implementação do curto prazo — tradução livre:

"são necessários mecanismos de *checkpoint* [...] para persistir o estado no nível da *thread*"

O que a nota 01 §8 gravou em quatro linhas é **memória de curto prazo persistida**.

A tese do slide anterior segue de pé, agora com o escopo dito:

O checkpoint não é uma memória **de longo prazo** deficiente. Onde esta aula escreve "memória" sem qualificar, leia-se **longo prazo**.

A objeção da janela grande

Uma janela de 1M *tokens* **não** substitui memória de longo prazo. A janela é reiniciada a cada requisição — uma janela maior compra mais **curto** prazo, e nunca longo prazo.

Necessidade	Por que a janela não atende
Persistir por dias e semanas	a sessão termina, e o conteúdo dela também
Aprender entre sessões	o da sessão anterior não está na requisição atual
Acesso seletivo	é preciso o fragmento relevante, não tudo o que ocorreu

E a Aula 05 (nota 03 §1) acrescenta o custo: o consumo acumulado cresce com o **quadrado** dos passos, e a qualidade cai antes de a janela acabar.

Parte 2

As três memórias

Três tipos, três estruturas

Tipo	Exemplo no domínio	Estrutura	Consulta
Episódica	"em 12/03 a D-4102 do F-088 foi aprovada"	índice vetorial	1 embedding
Semântica	"F-088 viaja para Portugal com frequência"	chave-valor	zero chamadas
Procedural	"quando o recibo diverge, devolver antes de analisar"	texto no <i>system prompt</i>	zero chamadas

A distinção só tem valor prático porque os três exigem **armazenamento diferente**.

Buscar um fato semântico por similaridade é caro e impreciso quando uma chave resolve.

Episódica — a única que se recupera por similaridade

```
class MemoriaEpisodica:
    def gravar(self, episodios):
        self.colecao.add(
            ids=[e["id"] for e in episodios],
            documents=[e["resumo"] for e in episodios],
            metadatas=[{"data": e["data"], "funcionario": e["funcionario"],
                        "veredito": e["veredito"]} for e in episodios],
            embeddings=gerar_matrix_embeddings(textos).tolist())
```

É a única em que a pergunta relevante — *"já ocorreu algo parecido?"* — é de **assunto**, que é o que o vetor mede.

Os três metadados vêm das cegueiras da Aula 06

Metadado	Cegueira que ele contorna
data	tempo — o vetor não tem noção de anterioridade
funcionario	entidade — ids do mesmo formato são indistinguíveis
veredito	negação — "aprovado" e "reprovado" a 0,94 de similaridade

```
def recuperar(self, consulta, k=3, funcionario=None, veredito=None):  
    """Filtra por metadado ANTES de ordenar por similaridade."""
```

A ordem importa. Filtrar depois de ordenar já perdeu o que o filtro resolveria.

Semântica e procedural

Semântica — fatos sobre entidades. Chave-valor, e a consulta é uma busca de dicionário. Custa zero chamadas.

Procedural — *como fazer algo*. Vai por inteiro no *system prompt*, e por isso é a que mais compete por janela.

"quando o recibo diverge do valor declarado, devolver antes de analisar o mérito"

Uma regra procedural quase sempre **nasce de um erro**. Ela é a memória de uma correção.

O que NÃO deve ser armazenado

Informação	Decisão
"a resposta veio com 1.820 tokens de entrada"	não guardar — ruído de execução
"o parecer P-1188 foi registrado para D-4612"	guardar — episódica com id
CPF, conta, credencial	não guardar — e a Aula 14 volta nisso
o que é recalculável	não guardar — é dívida

A parte mais importante da política de memória é a **lista de exclusão**. Ela é mais curta de escrever e mais difícil de lembrar.

O que fica

- Serializar tudo produz **arquivo morto**, não memória
- **Checkpoint** × **memória** × **RAG**: a origem do conteúdo decide tudo o mais
- *Short-term* é a janela; o **checkpoint** a persiste. *Long-term* é o resto da aula
- Memória exige **recuperar**, e recuperar exige **selecionar**
- Três tipos, **três estruturas** — e usar a errada é caro e impreciso
- Os metadados da episódica são as **defesas contra as cegueiras da Aula 06**

Próxima nota: **quem decide** o que entra na memória — e o que cada escolha custa.